

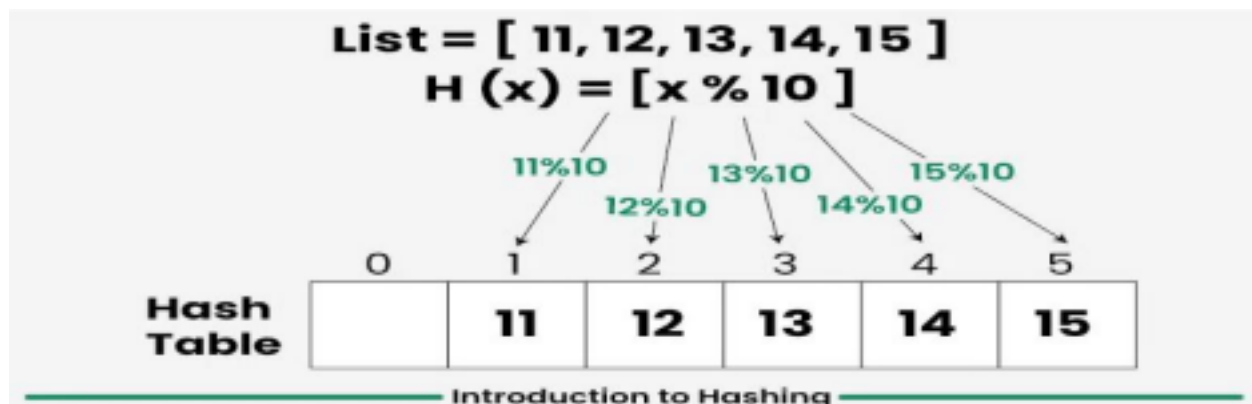
## Unit-5

### HASHING, MEMORY MANAGEMENT, AND REALWORLD APPLICATIONS

**Hashing and Hash Tables** – collision handling, load factor, real-world use in databases, **Disjoint Set (Union-Find)** – for connected components, social networks. **Memory management techniques and garbage collection logic**, Real world applications: **Real-time search** (e.g., Google autocomplete), **Social network graphs** (Facebook friend suggestions), **Fraud detection using graph cycles**, **Data structure use in OS/DBMS** (e.g., page replacement, indexing)

#### Introduction to Hashing and Hash Table:

**Hashing** refers to the process of generating a small sized output (that can be used as index in a table) from an input of typically large and variable size. Hashing uses mathematical formulas known as hash functions to do the transformation. This technique determines an index or location for the storage of an item in a data structure called Hash Table.



#### Hash Table Data Structure Overview

- It is one of the most widely used data structure after arrays.
- It mainly supports search, insert and delete in  $O(1)$  time on average which is more efficient than other popular data structures like arrays, Linked List and Self Balancing BST.
- We use hashing for dictionaries, frequency counting, maintaining data for quick access by key, etc.

- Real World Applications include Database Indexing, Cryptography, Caches, Symbol Table and Dictionaries.
- There are mainly two forms of hash typically implemented in programming languages.

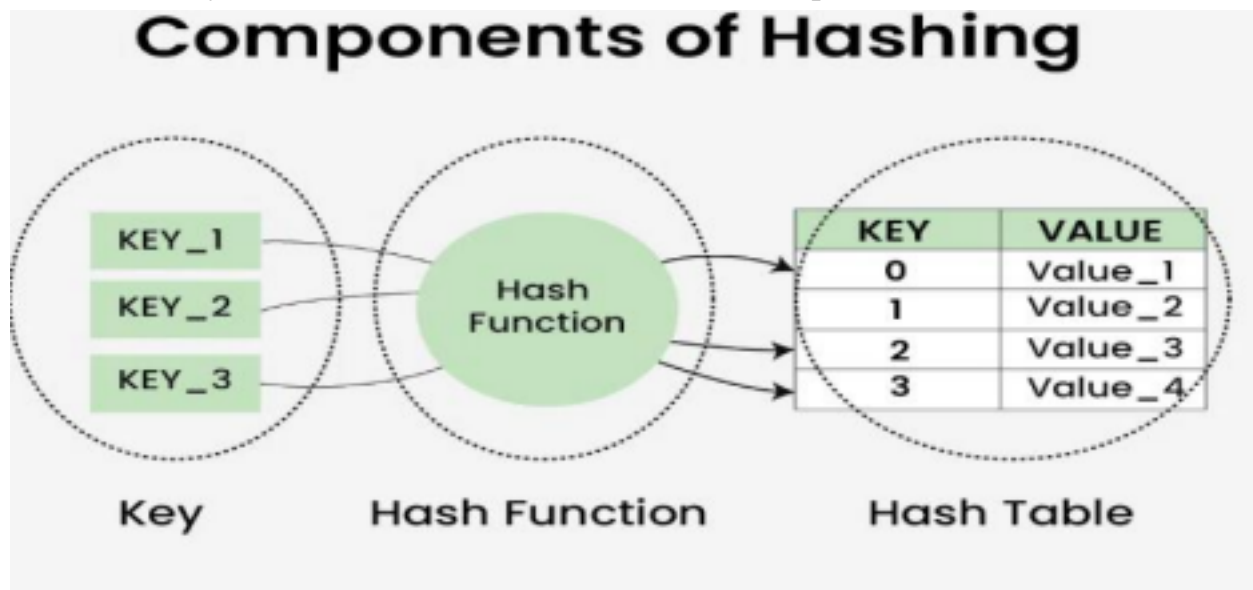
**Hash Set** : Collection of unique keys (Implemented as [Set in Python](#), [Set in JavaScript](#), [unordered\\_set in C++](#) and [HashSet in Java](#)).

**Hash Map** : Collection of key value pairs with keys being unique (Implemented as

## Components of Hashing

There are majorly three components of hashing:

1. **Key:** A Key can be anything string or integer which is fed as input in the hash function the technique that determines an index or location for storage of an item in a data structure.
2. **Hash Function:** Receives the input key and returns the index of an element in an array called a hash table. The index is known as the hash index .
3. **Hash Table:** Hash table is typically an array of lists. It stores values corresponding to the keys. Hash stores the data in an associative manner in an array where each data value has its own unique index.



**How does Hashing work?**

Suppose we have a set of strings {"ab", "cd", "efg"} and we would like to store it in a table.

- **Step 1:** We know that hash functions (which is some mathematical formula) are used to calculate the hash value which acts as the index of the data structure where the value will be stored.

- **Step 2:** So, let's assign

- "a" = 1,
- "b" = 2, .. etc, to all alphabetical characters.

- **Step 3:** Therefore, the numerical value by summation of all characters of the string:

- $"ab" = 1 + 2 = 3,$

- $"cd" = 3 + 4 = 7,$

- $"efg" = 5 + 6 + 7 = 18$

- **Step 4:** Now, assume that we have a table of size 7 to store these strings. The hash function that is used here is the sum of the characters in **key mod Table size** . We can compute the location of the string in the array by taking the **sum(string) mod 7** .

- **Step 5:** So we will then store

- "ab" in  $3 \bmod 7 = 3,$
- "cd" in  $7 \bmod 7 = 0,$  and
- "efg" in  $18 \bmod 7 = 4.$

## Mapping Key with indices of Array

| 0  | 1 | 2 | 3  | 4   | 5 | 6 |
|----|---|---|----|-----|---|---|
| cd |   |   | ab | egf |   |   |

The above technique enables us to calculate the location of a given string by using a simple hash function and rapidly find the value that is stored in that location. Therefore the idea of hashing seems like a great way to store (key, value) pairs of the data in a table.

### What is a Hash function?

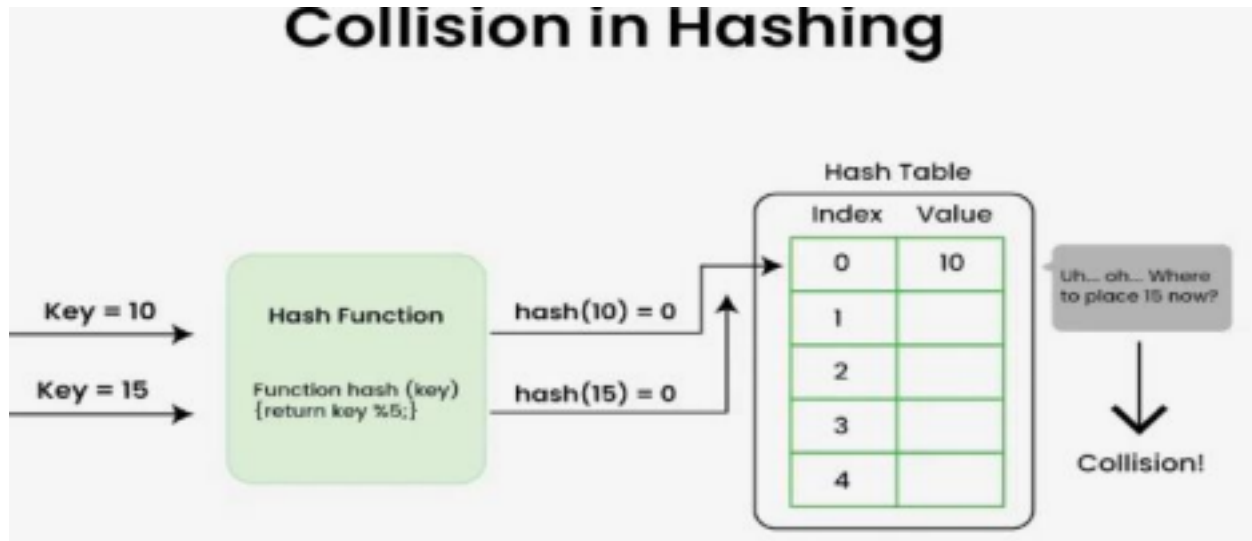
A [hash function](#) creates a mapping from an input key to an index in hash table, this is done through the use of mathematical formulas known as hash functions. For example: Consider phone numbers as keys and a hash table of size 100. A simple example hash function can be to consider the last two digits of phone numbers so that we have valid array indexes as output. A good hash function should have the following properties:

1. Efficient
2. Should uniformly distribute the keys to each index of hash table.
3. Should minimize collisions (This and the below are mainly derived from the above 2nd point)
4. Should have a low load factor (number of items in the table divided by the size of the table).

### What is Collision in Hashing?

When two or more keys have the same hash value, a **collision** happens. If we consider the above example, the hash function we used is the sum of the letters, but if we examined the hash function closely then the problem can be easily visualised that for different strings same hash value is being generated by the hash function.

For example: {"ab", "ba"} both have the same hash value, and string {"cd","be"} also generate the same hash value, etc. This is known as **collision** and it creates problem in searching, insertion, deletion, and updating of value.



The probability of a hash collision depends on the size of the algorithm, the distribution of hash values and the efficiency of Hash function. To handle this collision, we use [Collision Resolution Techniques](#).

### What is meant by Load Factor in Hashing?

The [load factor](#) of the hash table can be defined as the number of items the hash table contains divided by the size of the hash table. Load factor is the decisive parameter that is used when we want to rehash the previous hash function or want to add more elements to the existing hash table.

It helps us in determining the efficiency of the hash function i.e. it tells whether the hash function which we are using is distributing the keys uniformly or not in the hash table.

$$\text{Load Factor} = \text{Total elements in hash table} / \text{Size of hash table}$$

### What is Rehashing?

As the name suggests, [rehashing](#) means hashing again. Basically, when the load factor increases to more than its predefined value (the default value of the load factor is 0.75), the complexity increases. So to overcome this, the size of the array is increased (doubled) and all the values are hashed again and stored in the new

double

sized array to maintain a low load factor and low complexity.

## Hashing with Chaining Implementation

In [hashing](#) there is a hash function that maps keys to some values. But these hashing functions may lead to a collision that is two or more keys are mapped to same value. **Chain hashing** avoids collision. The idea is to make each cell of hash table point to a linked list of records that have same hash function value.

For a more detailed explanation and theoretical background on this approach, please refer to [Hashing | Set 2 \(Separate Chaining\)](#).

Let's create a hash function, such that our hash table has 'n' number of buckets.

To insert a node into the hash table, we first compute the hash index for the given key using a hash function:

**hashIndex = key % noOfBuckets.**

This index determines the appropriate bucket where the node should be inserted.

### Example:

noOfBuckets = 7

keys to insert = [15, 11, 27, 8]

For each key:

$$\cdot 15 \% 7 = 1$$

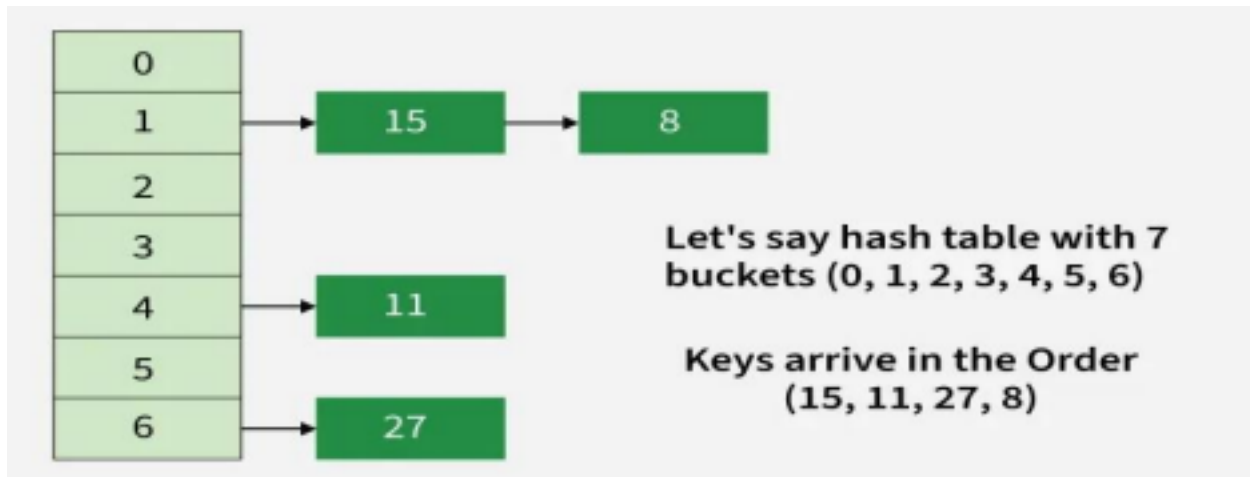
$$\cdot 11 \% 7 = 4$$

$$\cdot 27 \% 7 = 6$$

$$\cdot 8 \% 7 = 1$$

**Insert:** Move to the bucket corresponding to the above-calculated hash index and insert the new node at the end of the list.

**Delete:** To delete a node from hash table, calculate the hash index for the key, move to the bucket corresponding to the calculated hash index, and search the list in the current bucket to find and remove the node with the given key (if found).



### Linear Probing in Hash Tables

In [Open Addressing](#), all elements are stored in the hash table itself. So at any point, size of table must be greater than or equal to total number of keys (Note that we can increase table size by copying old data if needed).

- **Insert(k)** - Keep probing until an empty slot is found. Once an empty slot is found, insert k.
- **Search(k)** - Keep probing until slot's key doesn't become equal to k or an empty slot is reached.
- **Delete(k)** - Delete operation is interesting. If we simply delete a key, then search may fail. So slots of deleted keys are marked specially as "deleted".

Here, to mark a node deleted we have used **dummy node** with key and value -1. Insert can insert an item in a deleted slot, but search doesn't stop at a deleted slot. The entire process ensures that for any key, we get an integer position within the size

of the Hash Table to insert the corresponding value. So the process is simple, user gives a (key, value) pair set as input and based on the value generated by hash function an index is generated to where the value corresponding to the particular key is stored. So whenever we need to fetch a value corresponding to a key that is just  $O(1)$ .

## Linear Probing Example

|                                       |                                             |                                                   |                                                               |                                                                           |                                                                           |
|---------------------------------------|---------------------------------------------|---------------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Insert (76)                           | Insert (93)                                 | Insert (40)                                       | Insert (47)                                                   | Insert (10)                                                               | Insert (55)                                                               |
| $76\%7 = 6$                           | $93\%7 = 2$                                 | $40\%7 = 5$                                       | $47\%7 = 5$                                                   | $10\%7 = 3$                                                               | $55\%7 = 6$                                                               |
| 0<br>1<br>2<br>3<br>4<br>5<br>6<br>76 | 0<br>1<br>2<br>93<br>3<br>4<br>5<br>6<br>76 | 0<br>1<br>2<br>93<br>3<br>4<br>5<br>40<br>6<br>76 | 0<br>47<br>1<br>93<br>2<br>93<br>3<br>4<br>5<br>40<br>6<br>76 | 0<br>47<br>1<br>93<br>2<br>93<br>3<br>10<br>4<br>40<br>5<br>40<br>6<br>76 | 0<br>47<br>1<br>55<br>2<br>93<br>3<br>10<br>4<br>40<br>5<br>40<br>6<br>76 |

### Quadratic Probing in Hashing

[Hashing](#) is an improvement technique over the [Direct Access Table](#). The idea is to use a [hash function](#) that converts a given number or any other key to a smaller number and uses the small number as the index in a table called a [hash table](#).

### Quadratic Probing:

Quadratic probing is an [open-addressing](#) scheme where we look for the  $i^2$ 'th slot in the  $i$ 'th iteration if the given hash value  $x$  collides in the hash table. We have already discussed [linear probing implementation](#).

### How Quadratic Probing is done?

Let  $\text{hash}(x)$  be the slot index computed using the hash function. • If the slot  $\text{hash}(x) \% S$  is full, then we try  $(\text{hash}(x) + 1*1) \% S$ . • If  $(\text{hash}(x) + 1*1) \% S$  is also full, then we try  $(\text{hash}(x) + 2*2) \% S$ . • If  $(\text{hash}(x) + 2*2) \% S$  is also full, then we try  $(\text{hash}(x) + 3*3) \% S$ . • This process is repeated for all the values of  $i$  until an empty slot is found.

Let us consider a simple hash function as “key mod 7” and sequence of keys as 22,

30 and 50.



## COMMON COLLISION HANDLING TECHNIQUES

- Separate Chaining

- **How it works:** Each slot in the hash table stores a linked list, or another data structure, of all entries that have the same hash value.
- **Example:** If two keys hash to index 2, a linked list is created at index 2 to store both keys.

- Open Addressing

- **How it works:** When a collision occurs, the algorithm searches for the next available empty slot in the hash table.
- **Sub-techniques:**
  - Linear Probing: Checks slots sequentially until an empty slot is found.
  - Quadratic Probing: Checks slots at increasing intervals (e.g., 1, 4, 9, 16...) until an empty slot is found.
  - Double Hashing: Uses a second hash function to determine the step size for probing, helping to avoid primary clustering.

## Other Collision Contexts

Beyond hashing, "collision handling" can also refer to:

- Collision detection in computer graphics: The problem of detecting when

two or more virtual objects intersect.

- **Bus arbitration:** In some communication protocols, like CAN, the protocol can detect and resolve collisions without data loss by detecting when a node sends a high signal but reads a low signal on the bus.

Separate Chaining is a [collision handling technique](#). Separate chaining is one of the most popular and commonly used techniques in order to handle collisions. In this article, we will discuss about what is Separate Chain collision handling technique, its advantages, disadvantages, etc.

There are mainly two methods to handle collision:

- Separate Chaining
- Open Addressing

In this article, only separate chaining is discussed. We will be discussing Open addressing in the next post

### **Separate Chaining:**

The idea behind separate chaining is to implement the array as a linked list called a chain.

*Linked List (or a Dynamic Sized Array) is used to implement this technique. So what happens is, when multiple elements are hashed into the same slot index, then these elements are inserted into a singly-linked list which is known as a chain.*

Here, all those elements that hash into the same slot index are inserted into a linked list. Now, we can use a key K to search in the linked list by just linearly traversing. If the intrinsic key for any entry is equal to K then it means that we have found our entry. If we have reached the end of the linked list and yet we haven't found our entry then it means that the entry does not exist. Hence, the conclusion is that in separate chaining, if two different elements have the same hash value then we store both the elements in the same linked list one after the other.

**Example:** Let us consider a simple hash function as "**key mod 5**" and a sequence of keys as 12, 22, 15, 25



### Advantages:

- Simple to implement.
- Hash table never fills up, we can always add more elements to the chain.
- Less sensitive to the hash function or load factors.
- It is mostly used when it is unknown how many and how frequently keys may be inserted or deleted.

### Disadvantages:

- The cache performance of chaining is not good as keys are stored using a linked list. Open addressing provides better cache performance as everything is stored in the same table.
- Wastage of Space (Some Parts of the hash table are never used)
- If the chain becomes long, then search time can become  $O(n)$  in the worst case
- Uses extra space for links

## LOAD FACTOR:

### Definition

The load factor is a measure used in hash tables to determine the efficiency of the storage system, calculated as the ratio of the number of entries (or keys) in the hash table to the total number of slots (or buckets) available. It indicates how full a hash

table is, influencing both the likelihood of collisions and the performance of operations like insertion, deletion, and search. A higher load factor means more entries are stored in fewer slots, which can lead to increased collisions and decreased efficiency.

## 5 Must Know Facts For Your Next Test

1. The ideal load factor for hash tables is generally around 0.7 to 0.8, balancing space efficiency and performance.
2. When the load factor exceeds a certain threshold, it often triggers rehashing to resize the table and reduce the chances of collisions.
3. A load factor of 1 means that the hash table is perfectly filled, where each slot has exactly one entry, while values greater than 1 indicate that some slots are shared by multiple entries.
4. In practical terms, adjusting the load factor can significantly affect the average time complexity of operations: lower load factors usually result in faster average access times.
5. Maintaining an optimal load factor can also minimize memory usage, preventing wasteful allocation of space when it's not needed.

"Load factor" can mean different things depending on the context. Here are the main uses:

### 1. In Data Structures (Hash Tables)

• **Definition:** The ratio of the number of stored elements to the total number of available slots in the table.

• **Formula:**

$$\text{Load Factor} = \frac{\text{Number of elements}}{\text{Table size}}$$

- **Example:** If a hash table has 100 slots and 75 elements stored, the load factor is 0.75.
- **Purpose:** Helps measure how full the table is; higher load factor → more collisions → slower performance.

### ◆ What is Load Factor?

The **load factor** ( $\alpha$ ) in a hash table measures how full the table is.

It is defined as:

$$\alpha = \frac{n}{m}$$

- **n** = number of elements stored in the hash table
- **m** = total number of slots (buckets) in the table

### Load Factor Important?

- A **low load factor** (e.g., 0.2) → the table is mostly empty, collisions are rare, but space is wasted.
- A **high load factor** (e.g., > 0.7 or 0.8) → the table is nearly full, collisions are frequent, performance drops.

### ◆◆ Typical Threshold

- Many hash table implementations (like Java's HashMap) **resize the table** when the load factor exceeds **0.75**.
- This keeps operations like insert, search, and delete close to **O(1) average time complexity**.

### 💡 Example

Suppose:

- Hash table size  $m = 10$
- Elements inserted  $n = 7$

$$\alpha = \frac{7}{10} = 0.7$$

Since  $0.7 < 0.75$ , no resizing is needed yet. If one more element is inserted, the load factor becomes 0.8, which triggers resizing.

### 💡💡 Key Takeaways

- **Load factor** = fullness of hash table.
- Balances **time efficiency** (fewer collisions) with **space efficiency** (not too sparse).
- **Resizing strategy** is crucial to maintain good performance.

2. Real world use in database:

### Where Databases Use Load Factor

Databases often rely on **hash-based indexing** or **hash joins**, where **load factor** plays a key role in performance.

#### 1. Hash Indexing

- Many databases (e.g., PostgreSQL's **hash indexes**, MySQL's **MEMORY/HEAP tables**) use **hash tables** to quickly find rows.

• Example:

• `CREATE INDEX idx_user_email ON users USING HASH(email);` •

Here, a hash table maps each email to a slot pointing to a row (or page). •

**Load factor** determines:

- **Lookup speed:** If the table is too full (high  $\alpha$ ), collisions increase, making lookups slower.
- **Space efficiency:** Too low  $\alpha$  wastes memory.

**Real-world effect:** If millions of users register, and the load factor isn't controlled, queries like:

```
SELECT * FROM users WHERE email = 'abc@example.com';
```

could degrade from near **O(1)** lookups to **O(n)** scans in a bucket.

## 2. Hash Joins

- In query execution, **hash joins** use hash tables to match rows between two tables.
- Example:
- `SELECT *`
- `FROM orders o`
- `JOIN customers c ON o.customer_id = c.id;`
- The DBMS builds a hash table for the smaller table (say, customers).

**Load factor** affects:

- Join speed (more collisions = more probing per row).
- Memory usage (larger hash table = more RAM, but fewer collisions).

## Real world use in Data Base:

Databases are fundamental to nearly all modern applications and systems, serving as organized repositories for storing, managing, and retrieving data. Here are several real-world uses of databases:

### • E-commerce Platforms:

Online retailers like Amazon use databases to store product information (descriptions, prices, images), customer profiles, order history, payment details, and inventory levels, enabling smooth transactions and personalized shopping experiences.

### • Banking and Financial Services:

Banks rely on databases to manage customer accounts, transaction records, loan

details, and financial instruments. They are crucial for processing payments, detecting fraud, and generating financial reports.

- **Healthcare Systems:**

Databases are used in hospitals and clinics to store patient records, medical histories, appointment schedules, test results, and billing information, facilitating efficient patient care and administrative tasks.

- **Social Media Platforms:**

Platforms like Facebook and Instagram utilize databases to store user profiles, posts, photos, videos, messages, and friend connections, enabling real-time interaction and content sharing.

- **Supply Chain Management:**

Businesses use databases to track products from manufacturing to delivery, managing inventory, warehouse operations, transportation logistics, and vendor information.

- **Customer Relationship Management (CRM) Systems:**

CRM software like Salesforce uses databases to store and manage customer interactions, sales pipelines, marketing campaigns, and support tickets, helping businesses enhance customer engagement.

- **Educational Platforms:**

Databases are essential for managing student records, course information, instructor details, grades, and academic progress in schools, universities, and online learning platforms.

- **Project Management Tools:**

Databases underpin project management software, storing task lists, deadlines, team assignments, progress updates, and resource allocation information.

- **Government and Public Services:**

Government agencies use databases for managing citizen records, tax information, public services, and infrastructure data.

- **Retail Analytics:**

Databases are used in retail to analyze sales data, inventory levels, customer purchasing patterns, and promotional effectiveness to optimize business strategies.

## Distributed Databases & Partitioning

- Systems like Cassandra, DynamoDB, or MongoDB use **hashing for data partitioning/sharding**.
- Example: A user ID might be hashed to decide which server stores the data. •

**Load factor** matters here too:

- If the hash buckets (partitions) are overloaded, some servers handle more queries than others → **hotspots**.
- Keeping load factors balanced ensures fair distribution and high availability.

### ?? Real Example

- **PostgreSQL**: Default fill/load factor for B-tree indexes is ~90% (space efficiency focus). For **hash indexes**, the table resizes automatically when load factor gets too high.
- **Java HashMap in Hibernate (ORM)**: Hibernate internally uses Java's HashMap for caching entities. Load factor tuning (default 0.75) directly affects how fast entity lookups are.

Load factor in databases controls the tradeoff between **speed and space**

- in:
- Hash indexes (fast lookups on keys).
  - Hash joins (efficient query execution).
  - Partitioning/sharding (balanced distribution of data).

## 3.DISJOINT SET (UNION-FIND)

Two sets are called disjoint sets if they don't have any element in common. The disjoint set data structure is used to store such sets. It supports following operations:

- Merging two disjoint sets to a single set using Union operation. •

Finding representative of a disjoint set using Find operation.

- Check if two elements belong to same set or not. We mainly find representative of both and check if same.

Consider a situation with a number of persons and the following tasks to be performed on them:

- Add a new friendship relation, i.e. a person x becomes the friend of another person y i.e adding new element to a set.
- Find whether individual x is a friend of individual y (direct or indirect friend)

• Examples:

• *We are given 10 individuals say, a, b, c, d, e, f, g, h, i, j*

• *Following are relationships to be added:*

*a <-> b*

*b <-> d*

*c <-> f*

*c <-> i*

*j <-> e*

*g <-> j*

- *Given queries like whether a is a friend of d or not. We basically need to create following 4 groups and maintain a quickly accessible connection among group items:*

*G1 = {a, b, d}*

*G2 = {c, f, i}*

*G3 = {e, g, j}*

*G4 = {h}*

## **Operations on Disjoint Set Data Structures:**

### **1. Find:**

- The task is to find representative of the set of a given element. The representative is always root of the tree. So we implement find() by recursively traversing the parent array until we hit a node that is root (parent of itself).

### **2. Union:**

- The task is to combine two sets and make one. It takes **two elements** as

input and finds the representatives of their sets using the **Find** operation, and finally puts either one of the trees (representing the set) under the root node of the other tree.

```
class UnionFind:

    def __init__(self, size):

        # Initialize the parent array with each
        # element as its own representative
        self.parent = list(range(size))

    def find(self, i):

        # If i itself is root or representative
        if self.parent[i] == i:
            return

        # Else recursively find the representative
        # of the parent
        return self.find(self.parent[i])

    def unite(self, i, j):

        # Representative of set containing i
        irep = self.find(i)

        # Representative of set containing j
        jrep = self.find(j)
```

```
# Make the representative of i's set  
# be the representative of j's set
```

```
self.parent[irep] = jrep
```

```
# Example usage
```

```
size = 5
```

```
uf = UnionFind(size)
```

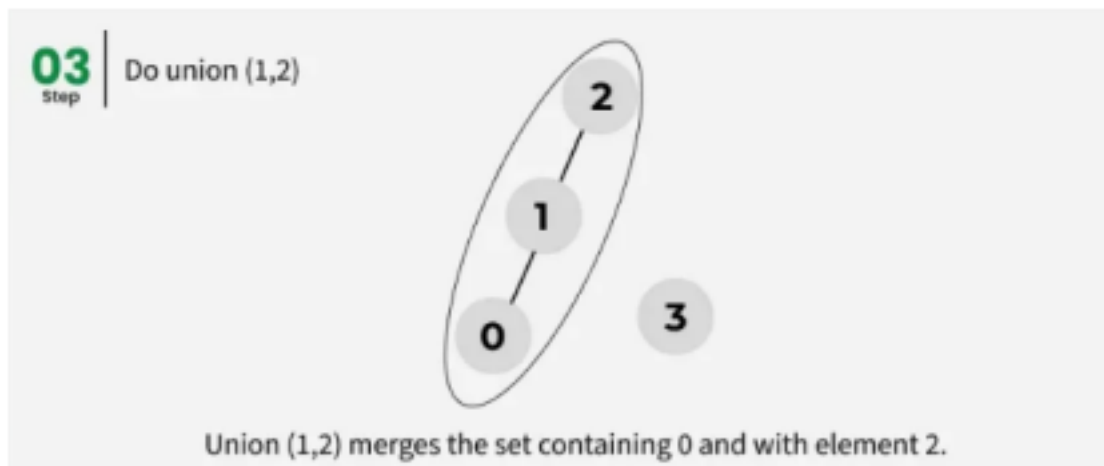
```
uf.unite(1, 2)
```

```
uf.unite(3, 4)
```

```
in_same_set = (uf.find(1) == uf.find(2))
```

```
print("Are 1 and 2 in the same set?", "Yes" if in_same_set else "No")
```

The above *union()* and *find()* are naive and the worst case time complexity is linear. The trees created to represent subsets can be skewed and can become like a linked list. Following is an example of worst case scenario.



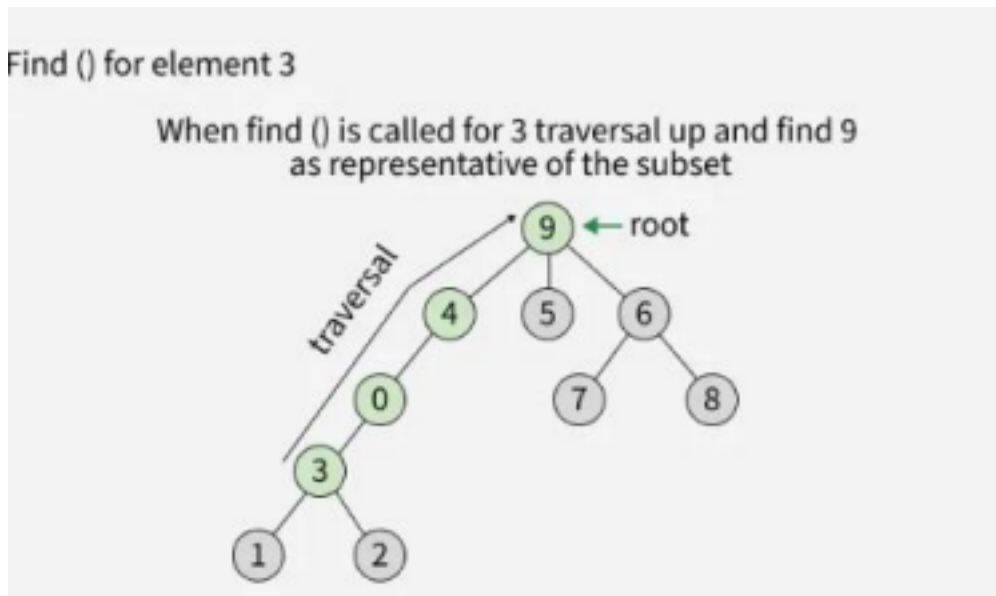
### Optimization (Path Compression and Union by Rank/Size):

The main idea is to reduce heights of trees representing different sets. We achieve this with two most common methods:

- 1) Path Compression
- 2) Union By Rank

### Path Compression (Used to improve find()):

The idea is to flatten the tree when *find()* is called. When *find()* is called for an element *x*, root of the tree is returned. The *find()* operation traverses up from *x* to find root. The idea of path compression is to make the found root as parent of *x* so that we don't have to traverse all intermediate nodes again. If *x* is root of a subtree, then path (to root) from all nodes under *x* also compresses.



It speeds up the data structure by **compressing the height** of the trees. It can be achieved by inserting a small caching mechanism into the **Find** operation. Take a look at the code for more details:

### What is a Disjoint Set?

A **Disjoint Set**, also called **Union-Find**, is a data structure that keeps track of a collection of **non-overlapping sets**.

It supports two main operations efficiently:

1. **Find(x)**: Determine which set an element *x* belongs to (usually by finding the “representative” or “parent” of the set).
2. **Union(x, y)**: Merge the sets containing *x* and *y*.

### Why Use It?

It's extremely useful when you need to keep track of connected components or group memberships.

Common applications:

- **Kruskal's algorithm** (to find Minimum Spanning Tree in graphs).
- **Cycle detection** in undirected graphs.
- **Network connectivity problems** (e.g., checking if two computers are in the same network).
- **Image segmentation, clustering**, and many others.

## Implementation

The naive way is inefficient, but two key optimizations make it very

fast: **1. Path Compression (for Find)**

- When you call `find(x)`, make every node on the path point directly to the root.
- This flattens the structure, making future finds faster.

## 2. Union by Rank/Size

- Always attach the smaller tree under the larger one when merging sets.
- This keeps the tree shallow.

## Time Complexity

With both optimizations:

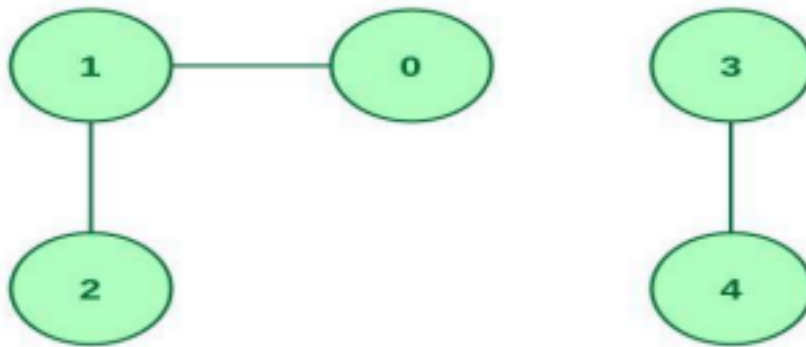
- **Find**  $\approx O(\alpha(n))$
- **Union**  $\approx O(\alpha(n))$

Here,  $\alpha(n)$  is the **inverse Ackermann function**, which grows so slowly that for all practical input sizes, it's almost constant

#### 4. Connected component definition & meaning in DSA

Connected component in an undirected graph refers to a group of vertices that are connected to each other through edges, but not connected to other vertices outside the group.

For example in the graph shown below,  $\{0, 1, 2\}$  form a connected component and  $\{3, 4\}$  form another connected component.



**Output:**  $[[0, 1, 2], [3, 4]]$

**Explanation:** There are 2 different connected components.

They are  $\{0, 1, 2\}$  and  $\{3, 4\}$ .

#### Characteristics of Connected Component:

- A connected component is a set of vertices in a graph that are connected to each other.
- A graph can have multiple connected components.
- Inside a component, each vertex is reachable from every other vertex in that component.

#### How to identify Connected Component:

There are several algorithms to identify Connected Components in a graph. The most popular ones are:

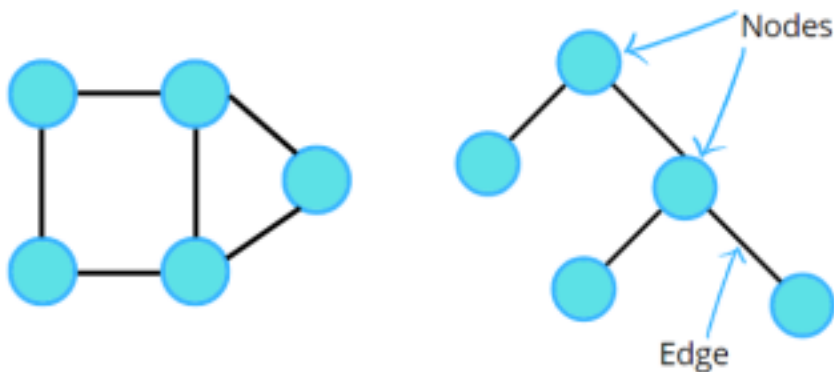
- [Depth-First Search \(DFS\)](#)
- [Breadth-First Search \(BFS\)](#)

- [Union-Find Algorithm \(also known as Disjoint Set Union\)](#)

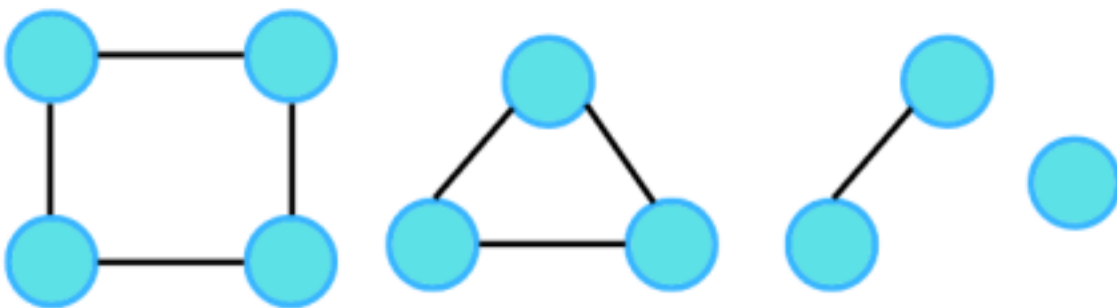
### Applications of Connected Component:

- [Graph Theory](#): It is used to find subgraphs or clusters of nodes that are connected to each other.
- [Computer Networks](#): It is used to discover clusters of nodes or devices that are linked and have similar qualities, such as bandwidth.
- [Image Processing](#): Connected components also have usage in image processing.

So far we've seen different types of graphs. Graphs can be connected or can be like a binary tree (as we know all trees are graphs with some restrictions) as shown in the following figure.



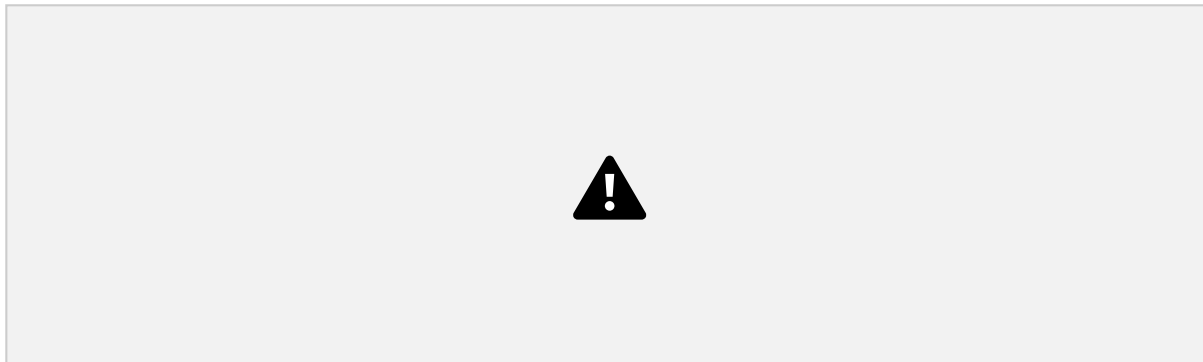
But what would you call the following figure?



The most common answer would be these are 4 different graphs as they are not connected.

But is it possible to call them a single graph? To answer this, let us consider the question given:

Given an undirected graph with 10 nodes and 8 edges. The edges are (1,2), (1,3), (2,4), (4,3), (5,6), (5,7), (6,7), (8,9). The graph that can be formed with the given information is as follows:



Apparently, it's a graph, which is in 4 pieces, the last one being a single node. In this case, we can say, the graph has been broken down into 4 different **connected components**. So next time if you see two different parts of a graph and they are not connected, then do not say that it cannot be a single graph. In the above example, they can be 4 different graphs but according to the given question and the input, we can call them parts of a single graph.

## 5.SOCIAL NETWORKS

Design data structures for a very large social network like Facebook or LinkedIn

How would you design the data structures for a very large social network like Facebook or LinkedIn? Describe how you would design an algorithm to show the shortest path between two people (e.g., Me-> Bob-> Susan-> Jason-> You).

**Case 1: Simplify the Problem (Not considering millions of people)** We can construct a graph by treating each person as a node and letting an edge between two nodes indicate that the two users are friends. If we want to find the path between two people, we start with one person and do a simple [breadth-first search](#). Alternatively, we can do bidirectional breadth first search. This means doing two breadth first searches, one from the source and one from the destination. When

the searches collide, we know we've found a path. Why not a depth-first search work well? First, the [depth-first search](#) would just find a path. It wouldn't necessarily find the shortest path. Second, even if we just needed any path, it would be very inefficient. Two users might be only one degree of separation apart, but it could search millions of nodes in their "subtrees" before finding this relatively immediate connection. In the implementation, we'll use two classes to help us. BFS Data holds the data needed for a breadth-first search, such as the Visited hash table and the to Visit queue. Path Node represents the path as we're searching, storing each Person and the previous Node we visited in this path.

### **How fast is above BFS based solution?**

Suppose every person has  $k$  friends, and Source  $S$  and Destination  $D$  have a friend  $C$  in common.

1. Traditional breadth-first search from  $S$  to  $D$ : We go through roughly  $k+k*k$  nodes: each of  $S$ 's  $k$  friends, and then each of their  $k$  friends.
2. Bidirectional breadth-first search: We go through  $2k$  nodes: each of  $S$ 's  $k$  friends and each of  $D$ 's  $k$  friends. Of course,  $2k$  is much less than  $k+k*k$ .
3. Generalizing this to a path of length  $q$ , we have this:

3.1 BFS:  $O(k^q)$

3.2 Bidirectional BFS:  $O(k^{q/2} + k^{q/2})$ , which is just  $O(k^{q/2})$

If we imagine a path like  $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$  where each person has 100 friends, this is a big difference. BFS will require looking at 100 million ( $100^4$ ) nodes. A bidirectional BFS will require looking at only 20,000 nodes ( $2 \times 100^2$ ).

### **Case 2: Handle Millions of Users**

For these many users, we cannot possibly keep all of our data on one machine. That means that our simple Person data structure from above doesn't quite work our friends may not live on the same machine as we do. Instead, we can replace our list of friends with a list of their IDs, and traverse as follows: 1: For each friend ID: `int machine_index = getMachineIDForUser(person_ID);` 2: Go to machine `#machine_index`

3: On that machine, do: `Person friend = getPersonWithID( person_ID);` The code below outlines this process. We've defined a class `Server`, which holds a list of all the machines, and a class `Machine`, which represents a single machine. Both classes have hash tables to efficiently lookup data.

**Following are some optimizations and follow-up questions. Optimization: Reduce machine jumps**

Jumping from one machine to another is expensive. Instead of randomly jumping from machine to machine with each friend, try to batch this jumps-e.g., if five of my friends live on one machine, I should look them up all at once.

**Optimization: Smart division of people and machines**

People are much more likely to be friends with people who live in the same country as they do. Rather than randomly dividing people across machines, try to divide them by country, city, state, and so on. This will reduce the number of jumps.

**Question: Breadth-first search usually requires "marking" a node as visited. How do you do that in this case?**

Usually, in BFS, we mark a node as visited by setting a visited flag in its node class. Here, we don't want to do that. There could be multiple searches going on at the same time, so it's a bad idea to just edit our data.

Instead, we could mimic the marking of nodes with a hash table to look up a node id and determine whether it's been visited.

```
from collections import deque
```

```
def bidir_shortest_path(adj, source, target):
```

```
    """
```

```
    adj: dict {node: set/list of neighbors}
```

```
    source, target: node ids
```

```
    returns: list of node ids representing shortest path or None if no path
```

```
    """
```

```
    if source == target:
```

```
        return [source]
```

```
    if source not in adj or target not in adj:
```

```
return None
```

```
# frontiers: node -> parent
```

```
parents_fwd = {source: None}
```

```
parents_bwd = {target: None}
```

```
frontier_fwd = set([source])
```

```
frontier_bwd = set([target])
```

```
# visited sets for quick test
```

```
visited_fwd = set([source])
```

```
visited_bwd = set([target])
```

```
while frontier_fwd and frontier_bwd: #
```

```
always expand the smaller frontier if
```

```
len(frontier_fwd) <= len(frontier_bwd):
```

```
frontier = set()
```

```
for u in frontier_fwd:
```

```
for v in adj.get(u, []):
```

```
if v not in visited_fwd:
```

```
parents_fwd[v] = u
```

```
visited_fwd.add(v)
```

```
frontier.add(v)
```

```
# check intersection
```

```

if v in visited_bwd:
    return _reconstruct_path(v, parents_fwd, parents_bwd)
frontier_fwd =
frontier

else:
    frontier = set()
    for u in frontier_bwd:
        for v in adj.get(u, []):
            if v not in visited_bwd:
                parents_bwd[v] = u
                visited_bwd.add(v)
                frontier.add(v)
            if v in visited_fwd:
                return _reconstruct_path(v, parents_fwd, parents_bwd)
frontier_bwd =
frontier

```

```

return None # no path found

```

```

def _reconstruct_path(meet, parents_fwd,
parents_bwd): # build path from source to meet
    path_fwd = []
    node = meet
    while node is not None:
        path_fwd.append(node)
        node = parents_fwd.get(node)

```

```

path_fwd.reverse() # now from source to meet

# build path from meet to target (skip meet to avoid duplication)
path_bwd = []

node = parents_bwd.get(meet)
while node is not None:
    path_bwd.append(node)
    node = parents_bwd.get(node)

return path_fwd + path_bwd

```

## 6.MEMORY MANAGEMENT TECHNIQUE AND GARBAGE COLLECTION LOGIC

### Memory Management and Garbage Collection

Heap allocation is a necessity for modern programming tasks, and so is automatic reclamation of heap-allocated memory. However, memory management comes with significant costs and has implications for how we generate code.

One way to avoid the cost of heap allocation is to stack-allocate objects. If an object allocated during execution of some function always becomes unreachable when the function returns, the object can be allocated on the function's stack frame. This requires an escape analysis to ensure it is safe. In fact, we can view the escape analysis used to stack-allocate activation records in languages with higher-order functions as an important special case of this optimization.

If an object will never be used again, it is safe to reclaim it. However, whether an object will be used again is an undecidable problem, so we settle for a conservative approximation. An object is **garbage** if it cannot be reached by traversing the object graph, starting from the immediate references available to the current computation(s): registers, stack, and global variables.

A garbage collector automatically identifies and reclaims the memory of these

garbage objects.

## Allocation and memory management

### Linear allocation

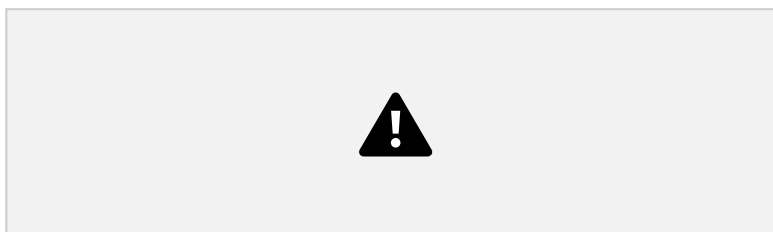
To collect garbage, we first have to create it. One strategy for heap management is **linear allocation**, in which the heap is organized so that all used memory is to one side of a next pointer, as shown below. An allocation request of bytes is satisfied by bumping up the next pointer by and returning its previous value. If the next pointer exceeds the limit pointer, the non-garbage objects are compacted into the first part of the memory arena so allocation can resume.

Linear allocation makes allocation very cheap, and it also tends to allocate related objects near each in memory, yielding good locality. For functional languages, which do a lot of allocation, it is a good choice. It is commonly used with **copying collectors**, which perform compaction.

Linear allocation

### Freelist allocation

Another standard organization of memory is to manage free blocks of memory explicitly. The free memory blocks form a linked list in which the first word of each block points to the next free block (it is therefore an **endogenous** linked list). An allocation request is satisfied by following the freelist pointer and finding a block big enough to hold the requested object.



Freelist, with allocated objects shown in gray

The basic freelist approach wastes space because of two problems: it has **internal fragmentation** in which objects may be allocated a larger block of memory than they requested. And it has **external fragmentation** in which the available memory is split among free memory blocks that are too small to accommodate a request.

We can speed up allocations and reduce internal fragmentation by having multiple freelists indexed by the size of the request. Typically small sizes are covered densely, and sizes grow geometrically after some point. Note, however, that when sizes grow as powers of two, on average about 30% of the space is wasted.

External fragmentation can be reduced by merging adjacent free blocks. This requires additional space for each memory block to keep track of its neighbors. The space may be in the memory block itself, or in an external data structure that keeps track of whether neighbors are in use (e.g., as a bitmap). The Berkeley allocator's “buddy system” is a neat example of the latter, with power-of-two sized memory blocks that can be merged to form a block of the next size up. The standard Linux malloc does not use a buddy system, however.

### **Automatic garbage collection**

It is common to expect the language to collect garbage for us. This task is more complex than we might expect, and modern language implementations contain sophisticated garbage collectors. Garbage collectors can do a better job if they can take advantage of help from the compiler, so it is worth understanding how GC and compilers interact.

An ideal garbage collector would have the following

attributes:

- Safe: only garbage is reclaimed.

- Precise: almost all garbage is collected, and reasonably soon.

Fully automatic.

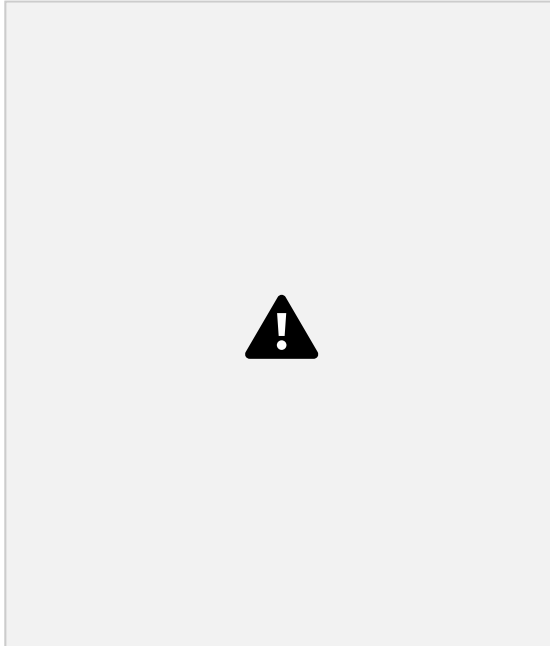
- Low overhead in time and space.

- Short **pause times**, in which the program waits for the collector.

Parallel: able to take advantage of additional cores.

- Simple.

The length of this list of requirements suggests that it might not be easy to achieve all of them simultaneously—which is indeed the case!



Mutator and GC. Garbage objects shown in gray.

Abstractly, we can think of a garbage collector as a thread or threads running perhaps concurrently with the threads doing useful computation. This situation is depicted above. We refer to the threads doing real computations as the **mutator**, since they are changing the object graph that the garbage collector is trying to manage.

### Reference counting

A standard (and often incorrectly implemented) approach to automatic memory management is **reference counting**. This is a simple local technique for collecting garbage. It is usually avoided because of its high run-time overhead, though recent work has argued that its overhead can be made comparable with other GC techniques, and it seems to be coming back into vogue.

Each object has a reference count approximating the number of incoming pointers to the object. For example, in the figure above, the leftmost gray object has no incoming pointers and a reference count of zero. The other two objects have a reference count of one each. We can already see a big problem with reference counting: it does not deal with cycles in the object graph, and must be combined with another garbage collection method if cycles are to be reclaimed.

The overhead of writes to update reference counts can be reduced by avoiding

updating reference counts. The key to this optimization is to notice that it is not necessary that the reference count exactly match the number of incoming references at all times. It is enough to enforce a weaker invariant:

- For safety, the reference count should only be decremented to zero if there are no incoming references.
- For precision, an object with no incoming references should eventually have a reference count of zero.

### **Garbage collection via traversal**

To identify cycles, we need an algorithm that traverses the object graph. Such algorithms, of which there are many, can be viewed as instances of Dijkstra's tricolor graph traversal algorithm.

In this algorithm, there are three kinds (or colors) of objects. **White** objects are the unvisited objects that have not yet been reached in the traversal. **Gray** objects have been reached, but may have successor objects that have not been reached. **Black** objects are completed objects. The key invariant of the algorithm is that black objects can never point to white objects. The algorithm starts with all nodes colored white. It then proceeds as follows:

1. The roots of garbage collection are colored gray.
2. While a gray object exists:
  - Color any white successors of gray.
  - Color black.
3. All objects are now black or white, and by the invariant, white objects are not reachable from black objects, and are therefore garbage. Reclaim them.
4. Optionally compact black objects.

### 7. Real world Application; Real Time Search (Google auto complete)

Google Search Autocomplete is a feature that predicts and suggests search queries

as users type into the search bar. As users begin typing a query, Google's autocomplete algorithm generates a dropdown menu with suggested completions based on popular searches, user history, and other relevant factors.

- In this article, we'll discuss the high-level design of Google's Search Autocomplete feature. This functionality predicts and suggests search queries as users type, enhancing the search experience.
- We'll explore the architecture, components, and challenges involved in building a scalable and efficient autocomplete system. Understanding Google's approach can provide valuable insights for developers and engineers working on similar systems.

### Functional Requirements for Google's Search Autocomplete

- **Instant Match Ideas:** As you type, the auto-fill should instantly show matching ideas. This makes the experience smooth and fast.
- **Accurate and Fitting:** The suggested ideas should be precise and make sense for what you've typed so far. Smart math does this by figuring out what you might want.
- **Customized Guesses:** The auto-fill should use info like your location, past searches, and popular topics. This way, its guesses fit you specifically.
- **Data Handling Made Easy:** Google needs to store and access many user searches and suggestions quickly. It should have great ways to save and find this data fast.

### Non-Functional Requirements for Google's Search Autocomplete

- **Speed Matters:** The autocomplete tool must work super fast. When you start typing, suggestions should pop up right away, even if you're far from Google's home base.
- **You Can Count On It:** Autocomplete needs to be reliable. It should always work properly so you can get accurate suggestions without interruptions or downtime.
- **Many Users, No Problem:** Lots of people use Google at once. The system

must handle many users smoothly, keeping everything running smoothly during busy times.

- **Global Scale:** The autocomplete system should give speedy and fitting answers worldwide. It should work well for people from different places and languages. But it must act the same way and be right all the time.
- **Security and Privacy:** The system must keep user details and privacy safe. It should handle search queries and suggestions securely. And it must follow rules and privacy policies.
- **Adaptability and Evolution:** The system should change as user habits, search trends, and tech move forward. Updates and improvements will make it better for users. This helps the system stay ahead in the search engine market.

## Capacity Estimation for Google's Search Autocomplete

### Traffic Estimations for Google's Search Autocomplete

- **User Traffic (UT):** This is the total number of searches Google receives per day globally. Let's assume this to be 3 billion searches per day.
- **Queries per User (QPU):** This represents the average number of searches performed by a user in a single session. Let's assume a user performs 3 searches in one session.
- **Average Session Duration (ASD):** This is the average time a user spends in a single search session. Let's assume this to be 5 minutes.
- **Queries per Second (QPS):** This is the average number of searches Google receives per second. It's calculated based on the total number of searches per day, divided by the number of seconds in a day.  **$QPS = (\text{User Traffic} \times \text{Queries per User}) / \text{Seconds in a Day}$**

Let's calculate QPS using the provided assumptions:

$$UT = 3 \times 10^9 \text{ searches/day}$$

$$QPU = 3 \text{ searches/session}$$

$$ASD = 5 \text{ minutes} = 5/60 \text{ hours}$$

$$\text{Seconds in a Day} = 24 \times 60 \times 60 = 86,400 \text{ seconds}$$

**Plugging in these values:  $QPS = 3 \times 109 \times 386,400$   $QPS = 86,4003 \times 109 \times 3$   
 $QPS \approx 104,167$  queries/second**



### **1. Clients:**

- These are the end-users or applications that interact with the autocomplete system to get search suggestions.
- Clients send search queries to the API Gateway for processing and receive autocomplete suggestions in response.

### **2. [API Gateway](#):**

- The API Gateway acts as an entry point for clients to access the autocomplete system.
- It receives incoming requests from clients, routes them to the appropriate backend services, and handles authentication, authorization, and rate limiting.
- In contrast to Google Search Autocomplete, the API Gateway in your system design abstracts the internal components and provides a unified interface for clients to interact with the system.

### **3. [Load Balancer](#):**

- Load balancers distribute incoming client requests across multiple instances of the suggestion service to ensure scalability, fault tolerance, and optimal resource utilization.
- They monitor the health of backend servers and route traffic accordingly.

- In the context of Google Search Autocomplete, load balancers ensure that requests are evenly distributed among suggestion service instances to handle varying loads efficiently.

#### **4. Suggestion Service:**

- The suggestion service is the core component responsible for generating autocomplete suggestions based on incoming search queries.
- It processes queries, retrieves relevant suggestions from the data store (Redis cache or NoSQL Trie data servers), and returns them to the client via the API Gateway.
- This service may incorporate various algorithms and data structures to efficiently retrieve and rank suggestions.
- In contrast to Google Search Autocomplete, the suggestion service in the design is responsible for both fetching suggestions and serving them to clients.

#### **5. Redis Cache:**

- Redis cache is an in-memory data store used to cache frequently accessed search queries and their corresponding autocomplete suggestions.
- It helps reduce latency by storing precomputed results and serving them quickly to clients without hitting the backend data store.
- Redis cache improves performance by caching frequently requested suggestions, reducing the load on the backend suggestion service and improving overall system responsiveness.

#### **6. NoSQL Trie Data Servers:**

- NoSQL Trie data servers store the trie data structure used for efficient prefix matching and search.
- They maintain a distributed, scalable database of search queries organized in a trie format, allowing for fast lookup of autocomplete suggestions.
- These servers store the raw trie data structure, enabling efficient retrieval of

suggestions without needing to compute them on the fly.

## **7. Snapshots Database:**

- The snapshots database stores periodic snapshots or backups of the system's data for disaster recovery, backup, and archival purposes.
- It ensures data integrity and provides a fallback mechanism in case of data loss or corruption.
- In contrast to Google Search Autocomplete, the snapshots database in your system design ensures data durability and availability, enabling the system to recover from failures and maintain consistent data across different components.

## **8. Zookeeper:**

- Zookeeper is a centralized service for maintaining configuration information, providing distributed synchronization, and facilitating coordination among distributed systems.
- It helps manage distributed resources, elect leaders, and maintain consensus in a distributed environment.
- In contrast to Google Search Autocomplete, Zookeeper ensures the coordination and consistency of distributed components in your system design, such as load balancers, suggestion services, and data servers, enabling them to work together seamlessly and maintain system integrity

## **Scalability for Google's Search Autocomplete**

More people using the system means more traffic. To handle the extra load, the system can add more servers. These servers help spread out the traffic. Load balancers make sure the traffic is shared evenly across all servers. The system also stores data that people ask for often. Storing this data means the servers don't have to get it from storage every time. Separate databases and microservices also let the system easily grow as more people use it.

Scalability in Google's search autocomplete is achieved through:

- **Horizontal Scaling:** More servers share the traffic load across them. Simply

put, adding extra computers to deal with a lot of people using your website or app.

- **Load Balancers**: Even distribution of online visitors, so no server gets overloaded, is done via 'load balancers' -- clever systems managing traffic flow.
- **Caching**: Frequently used data gets stored temporarily, called 'caching'.  
Reduces database workload, makes your experience faster.
- **Distributed Databases and Microservices**: Breaking down an application into mini-services handling specific tasks is called 'microservices'. Databases too become distributed for efficient scaling.
- **Asynchronous Processing and Message Queues**: Time-taking jobs get pushed to separate 'queues'. While you wait, the main system stays responsive, not hanging or crashing.
- **Auto-scaling**: Resources like servers automatically increase or decrease based on real-time usage demands through 'auto-scaling' -- optimizing both performance and costs.

## **8. SOCIAL NETWORK GRAPH:(FACE BOOK FRIEND SUGGESTIONS)**

Social networks are more than just connections between people. They are dynamic systems of users, content, and interactions. On platforms like Twitter, Reddit, or LinkedIn, relationships form not only between individuals but also between people and the content they create, share, and respond to. These interactions can be modeled using a powerful abstraction: the graph.



Figure: A social network graph representation

A **social network graph** represents entities such as users, posts, or comments as **nodes**, and the relationships between them, such as follows, replies, or likes, as **edges**. This structure allows us to study how influence spreads, how communities form, and which users or content serve as bridges between otherwise separate groups.

In this post, we explore how to model social media platforms as graphs, explain the key **mathematical metrics** used to analyze them, and examine practical tools that help turn large-scale social data into actionable insight. Whether you're trying to detect viral content, map influence, or understand user behavior, social network graphs provide the analytical foundation.

### What Is a Social Network Graph?

A social network graph is best represented using the **labeled property graph model**, which captures both the structure and semantics of social systems.

In this model:

- **Nodes** represent entities such as users, posts, comments, or groups.
- **Edges** represent relationships or interactions between those entities, such as follows, replies, likes, or group memberships.

- **Labels** are used to distinguish node and edge types (e.g., User, Post, FOLLOWS, LIKES).
- **Properties** can be attached to both nodes and edges—such as timestamps, content, or weights—to support deeper analysis.



Figure: Example of a simple social network

This representation is widely adopted in modern graph databases and query engines because it allows for flexible, expressive modeling of real-world data. It's especially well suited for social networks, where the diversity of entities and relationships goes far beyond simple friendships.

### What Kinds of Nodes and Edges Exist in Social Networks?

Modern social platforms involve many types of entities:

- **Users or accounts**
- **Posts** (e.g., tweets, videos, status updates)
- **Comments or replies**
- **Groups or communities**
- **Hashtags or topics**

Edges can represent actions or relationships like:

- A User *follows* another User
- A User *writes* a Post • A User *likes* a Post • A Comment *replies to* another Comment • A User *joins* a Group

This diversity makes social network graphs heterogeneous in structure, but the labeled property graph model handles it naturally: different node types are simply labeled nodes; different relationships are labeled edges.

### Graph Directionality and Weights

The nature of an edge may vary depending on the relationship it models:

- **Directed edges** for asymmetric relationships (e.g., FOLLOWS, REPLIES\_TO)
- **Undirected edges** for mutual ones (e.g., FRIENDS\_WITH)
- **Weights** to capture frequency or strength of interaction (e.g., number of messages exchanged)

These structural details affect how the network is interpreted and how graph algorithms behave.

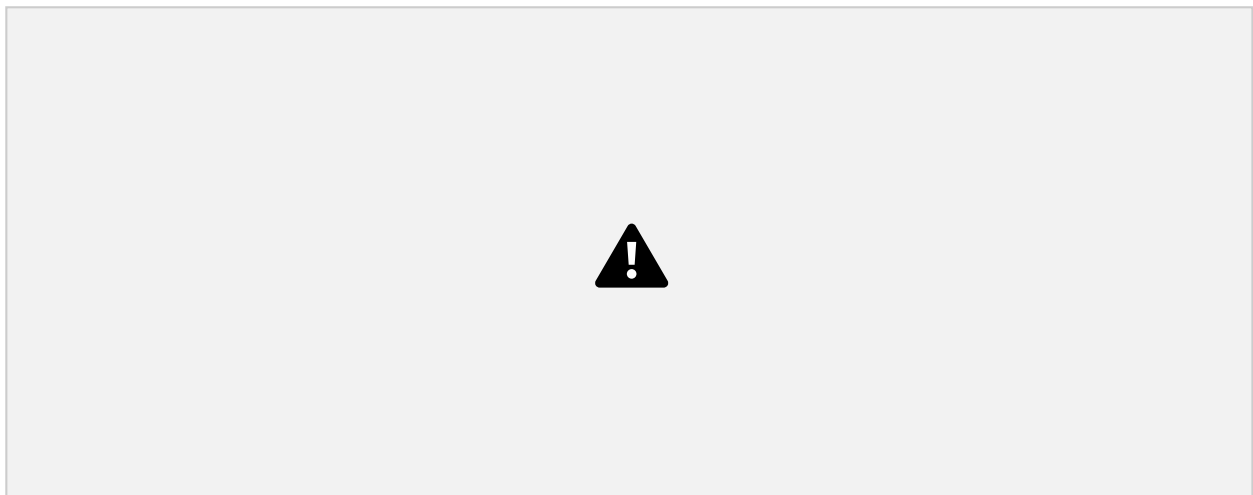


Figure: An example of a labeled property graph for social networks.

By grounding social network graphs in a flexible, expressive model, we enable

both precise structural representation and powerful analytical possibilities.

## **Structural Patterns in Social Network Graphs**

Social network graphs are not randomly connected. They tend to exhibit structural patterns that reflect the behavior of individuals and groups within a society or platform. Understanding these patterns helps reveal how influence works, how communities form, and how information moves through the network.

## **Social Network Graph Metrics and What They Reveal**

To understand the structure and behavior of a social network graph, we need more than visual inspection. We need quantitative measures. Social network metrics are mathematical tools that describe how central, connected, or influential a node is, and how the network behaves as a whole. These metrics are foundational in identifying key users, mapping influence, detecting communities, and evaluating how information spreads.

**Degree centrality** is the simplest measure: it counts how many connections a node has. In a social graph, a high-degree user may be popular, active, or simply well known. For example, in a follower network, someone with thousands of incoming connections might be considered an influencer. However, degree alone doesn't capture how a user fits into the broader network.

**Closeness centrality** measures how close a node is to all others, based on the shortest path lengths. A user with high closeness centrality can reach others quickly, which often correlates with the ability to spread information or receive updates rapidly. This metric is useful when assessing accessibility or efficiency of communication within the network.

**Betweenness centrality** captures a different aspect: control over information flow. It measures how often a node appears on the shortest paths between other nodes. Users with high betweenness centrality act as bridges between different parts of the network. They may not have many direct connections, but they are strategically positioned to influence or monitor interactions between groups.

**Eigenvector centrality** and its variant, **PageRank**, take into account not just how many connections a node has, but how important those connections are. A node connected to other well-connected nodes will score higher. This recursive notion of

influence is especially valuable in ranking users, posts, or topics that gain visibility not merely by volume, but by proximity to other central entities.

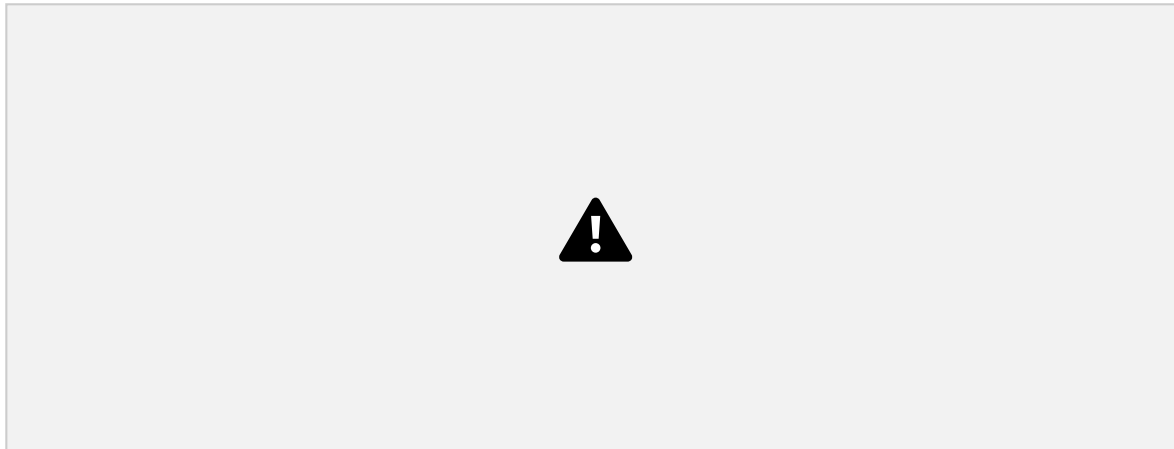


Figure: A simple illustration of the PageRank algorithm. The percentage shows the perceived importance, and the arrows represent hyperlinks. (Credited to [Wikipedia](#))

**Clustering coefficient** measures how likely it is that a user's neighbors are also connected to one another. A high clustering score suggests tight-knit communities or social circles. This is useful for identifying local cohesion and can help detect groups where information is likely to circulate but not escape.

Beyond node-level metrics, we can look at properties of the graph as a whole. **Connected components** identify isolated subgraphs—regions where users interact internally but have no ties to the broader network. **Graph density** measures how many connections exist compared to the maximum possible, offering a sense of how saturated the network is. The **diameter**, defined as the longest shortest path between any two nodes, gives an upper bound on how far information must travel.

Each of these metrics reveals something different: who is visible, who is strategic, who is clustered, and how the entire network behaves. In practice, they are often used together—for example, identifying high-betweenness users in low-density regions, or finding highly ranked posts that emerge from specific communities. These measurements are not just theoretical—they drive decisions in recommendation systems, trend analysis, moderation, and outreach strategies across real platforms.

## Tools and Techniques for Social Network Graph Analysis

Analyzing a social network graph requires more than knowing its structure. Once the graph is modeled, the challenge is to compute relevant metrics, explore subgraphs, and run queries that reflect meaningful social patterns. A range of tools and systems support this workflow, from low-level graph libraries to specialized engines and query languages.

The first category includes **graph analytics libraries** designed for metric computation. Libraries such as [NetworkX](#) (Python), [iGraph](#) (R, Python, C), and [GraphX](#) (Apache Spark) offer efficient implementations of standard algorithms like PageRank, betweenness centrality, community detection, and clustering coefficient. These tools are well suited for researchers or engineers who need full control over the analysis process and are comfortable writing code to define workflows. For large-scale processing, systems like GraphX and SNAP (from Stanford) support distributed computation across massive networks.

While graph libraries are powerful for analysis, they don't provide a convenient way to explore or interact with the graph structure in real time. That's where **graph query languages** come in. Languages like [openCypher](#) and [Gremlin](#) allow users to express complex relationship patterns, multi-hop traversals, and filtering conditions in a declarative way. For example, a query might ask for all users who liked posts written by users followed by Alice, or trace the shortest path between two hashtags in a discussion network. These languages are not designed to compute global metrics like PageRank directly, but they are essential for building, exploring, and refining graph-based investigations.

One of the most consistent patterns is the **heavy-tailed degree distribution**. In most social graphs, a small number of users have an exceptionally high number of connections—these might be celebrities, public figures, or highly active accounts. Meanwhile, the vast majority of users have only a handful of connections. This imbalance makes the network resilient in some ways but also highly dependent on its hubs for connectivity and visibility.

Another important feature is **clustering**, which captures the idea that people who are connected to the same person are more likely to be connected to each other. This phenomenon, known as **triadic closure**, leads to the formation of densely

linked groups or local neighborhoods. In graph terms, this is measured by the clustering coefficient, and it often reflects real-world social circles, such as families, friend groups, or coworkers.

Despite the presence of tight clusters, social networks as a whole often have **short average path lengths**. This is the essence of the **small-world effect**, where any two people are separated by only a few steps—even in networks with millions of nodes. The result is that ideas, trends, and influence can travel rapidly, even across seemingly distant parts of the network.

Finally, social graphs tend to have clear **community structure**. These communities consist of nodes that are more densely connected internally than with the rest of the graph. They may correspond to interest groups, shared identities, or coordinated activity. Community detection is a key task in social network analysis because it reveals the underlying organization of the network—who interacts with whom, and where the boundaries lie between different social spheres.

## 9.FRAUD DETECTION USING GRAPH CYCLES

Many enterprises still rely on fraud detection systems built for a different era—using static rules, anomaly thresholds, and siloed event scoring. These tools can catch obvious fraud, but they fall short against today’s coordinated, fast-moving attacks that span users, devices, accounts, and time.

A common misstep is assuming that more historical data will improve detection. But modern fraud isn’t just anomalous—it’s relational. Synthetic identities, shared infrastructure, and multi-entity behaviors are hard to spot unless you analyze how things connect.

Another misunderstanding is treating fraud and anti-money laundering (AML) as interchangeable. While both involve financial crime, fraud is real-time and loss driven, while AML is compliance-driven and retrospective. Effective graph solutions must address both, but with workflows tailored to their distinct timelines. Flattening fraud data into rows—one transaction at a time—strips away the structural context where signals often live. Fraud today isn’t just about a single bad actor. It’s about the behaviors, overlaps, and networks they exploit. Graph technology helps you see and act on those connections in real time.

## **What is a Fraud Detection Graph Database?**

A fraud detection graph database identifies suspicious behavior by analyzing the relationships between entities like users, accounts, transactions, devices, and IPs. These entities are modeled as nodes, with interactions (logins, payments, shared infrastructure) represented as edges.

This structure allows detection systems to ask smarter questions:

- Is this account one hop from a known fraud ring?

- Has this phone or email been reused across multiple applications? •

Are we seeing circular or layered transaction flows?

Unlike traditional systems that rely on slow joins or require data to be exported into external tools, TigerGraph performs these queries directly inside the graph—without flattening the data or introducing delays.

This is possible through native traversal, which means TigerGraph can quickly follow the web of relationships between accounts, devices, and transactions without reformatting the data or breaking context. Combined with in-graph computation and parallel execution across distributed systems, this allows TigerGraph to detect complex fraud patterns—like synthetic identities, laundering loops, and collusive networks—in real time, as behavior unfolds.

It's not just about visualization. It's computation. Graph-based detection surfaces the deeper story behind the risk.

## **Why Use a Fraud Detection Graph Database?**

Fraud doesn't happen in isolation—it spreads through connections. Traditional tools focus on individual transactions or users, flagging anomalies based on thresholds or patterns in isolation. But modern fraud is collaborative, distributed, and designed to look normal, one event at a time.

Graph technology uncovers the structure behind the behavior. Instead of looking at what a single actor does, it reveals how that actor is connected—and to whom. That makes it possible to surface:

- Collusion: Groups of accounts transacting in coordinated patterns, often sharing IPs, devices, or referral codes.
- Synthetic identities: Clusters of applications or users with overlapping credentials or infrastructure.
- Obfuscation tactics: Transaction loops and chains that obscure the origin and destination of funds.
- Behavioral proximity: New users who behave like known fraudsters, even if they appear unrelated on the surface.

These patterns are invisible to systems built around flat data. Even machine learning models struggle when the fraud isn't in the row—it's in the network.

TigerGraph is purpose-built for this type of detection. Its key capabilities include:

- Multi-hop traversal: Instantly trace behavior across multiple entities, revealing how a device or transaction links back to broader risk patterns.
- Sub-second performance: Run deep, multi-hop queries across graphs with billions of edges.
- In-graph computation: Score risk, detect structures, and run algorithms in place—no need for external joins or exports.

With graph, you don't just flag suspicious behavior—you understand where it fits in a larger structure. That context turns scattered alerts into actionable intelligence, and helps teams move from reactive investigations to proactive prevention.

## **Key Use Cases for Fraud Detection Graph Database**

Graph-based detection shines in scenarios where fraud is subtle, coordinated, or spread across multiple entities. Instead of chasing isolated anomalies, it helps teams surface the underlying structure—making it possible to detect fraud before it scales.

### *Synthetic Identity Fraud*

Fraudsters often create synthetic profiles by combining real and fake information. Individually, these profiles pass verification. But graph reveals shared

infrastructure like reused phone numbers, IPs, or devices—highlighting identity webs designed to game the system.

### *Transaction Laundering*

Illicit merchants may disguise payments by routing them through seemingly unrelated businesses. Graph exposes circular flows, high-frequency paths, and indirect connections that suggest laundering—even when individual transactions appear legitimate.

### *Collusive Networks*

Fraud doesn't just happen between individuals. Entire clusters of merchants or users may work together—sharing bank accounts, referring one another, or transacting in suspicious sequences. TigerGraph's community detection and similarity scoring flag these behavior clusters fast.

### *Account Takeover (ATO)*

When fraudsters gain unauthorized access to accounts, they often reuse devices, credentials, or behavioral patterns. Graph can link login attempts, detect cross account infrastructure reuse, and reveal access paths that mimic past breaches.

### *Claims and Insurance Fraud*

Repeated use of the same service providers, co-claimants, or vehicles across unrelated claims can indicate organized insurance fraud. Graph highlights overlapping participants and clustered behaviors that suggest coordination.

TigerGraph makes these use cases operational—not just analytical. With real-time traversal, in-graph pattern detection, and dynamic updates, teams don't just catch fraud after the fact—they anticipate it, trace its path, and intervene sooner.

## **Why It's Important**

Fraud isn't just about bad actors—it's about the structures they exploit. Attackers share infrastructure, reuse credentials, and distribute activity to avoid detection. Traditional tools, focused on single transactions or static rules, miss the web of connections that gives fraud its power.

Graph flips that model. It turns relationships into signals, revealing collusion, obfuscation, and behavioral mimicry that would otherwise go unnoticed.

Most systems treat each event—login, payment, account change—as an isolated data point. They flag anomalies but lack context. A suspicious transaction might trigger an alert, but it’s hard to judge true risk without understanding who else is involved or how behaviors cluster.

Graph-based detection doesn’t just show *what* happened—it shows *how* and *why*. It traces behavior through relationships, identifies coordination patterns, and highlights proximity to known threats. That makes it faster, more accurate, and more explainable.

With real-time scoring, analysts can stop fraud before it spreads. And with traceable paths and transparent scoring logic, investigations move faster—providing the context regulators, support teams, and legal teams require.

With TigerGraph, this isn’t a theory—it’s how enterprises move from slow reaction to proactive control.

## **Best Practices for Graph-Based Fraud Detection**

Graph fraud detection works best when it’s treated not as a bolt-on feature but as a core part of your fraud strategy. That means designing the graph to reflect how fraud behaves: across time, systems, and relationships.

### *Model for behavior over time*

Fraud isn’t static—it unfolds. A login today might connect to a transaction next week. A device could resurface on a new account next month. Your graph should capture this temporal evolution with timestamped edges, relationship recency, and frequency weighting.

### *Keep the graph current*

Fraud moves fast. Delays in data ingestion create blind spots. TigerGraph’s streaming ingestion lets you build a live fraud graph that syncs with API feeds, transactions, and login events—without batch delays or downtime.

### *Use multiple detection layers*

There’s no single algorithm for fraud. Combine techniques to expose different patterns:

- Similarity scoring reveals shared infrastructure (e.g., devices, IPs)

- PageRank or centrality identifies highly connected risk hubs •

Connected components isolate collusion rings

- Cycle detection flags transaction laundering and loops

TigerGraph includes all of these detection techniques out of the box, and they can be customized and combined using its built-in query language, GSQL—so teams can tailor the logic to fit their specific fraud patterns without needing to move data or rely on external tools.

*Make explainability part of the system* Regulators and internal teams need more than scores—they need to know why an account was flagged. Use subgraph visualizations and path tracing to explain why. With TigerGraph, every detection can be traced back to the nodes and edges that triggered it.

*Keep the data in the graph* Exporting data for offline scoring breaks context and adds latency. With TigerGraph, run all detection logic—including feature engineering, scoring, and alerting—inside the graph engine itself. That’s how you get real-time insights with full relational context intact.

Together, these practices turn graph technology from a visualization tool into a production-ready fraud detection engine—already powering real-time systems at scale across finance, insurance, and digital commerce.

## **Overcoming Challenges in Fraud Detection with Graph**

Graph-based fraud detection offers huge advantages—but it also introduces operational challenges. Many teams struggle to integrate graph into production, not because the value isn’t clear, but because their tools and data pipelines aren’t built for it. Here’s how TigerGraph addresses the most common blockers:

*Challenge 1: Fragmented data breaks detection* Fraud signals often live in silos—account logs, device histories, transactions, and customer records. Without a unified model, key connections are lost.

TigerGraph ingests data from multiple sources in real time—Kafka streams, REST APIs, relational systems—consolidating fragmented signals into a live, connected fraud graph.

*Challenge 2: Real-time performance doesn't scale* Fraud graphs grow fast. Most platforms slow to a crawl under multi-hop traversal, deep joins, or high concurrency.

TigerGraph's native parallel architecture supports distributed, multi-hop queries across billions of edges in milliseconds. There is no flattening or preprocessing—just real-time results at scale.

*Challenge 3: Alerts lack context* Legacy tools flag anomalies but rarely explain them. Analysts face long investigations with limited visibility into relationships.

Every risk score is connected to a path. TigerGraph analysts can trace behavior across accounts, devices, and transactions—visually and programmatically. This turns alerts into action.

*Challenge 4: Fraud and AML teams operate in silos* Fraud prevention requires real-time speed. AML compliance demands deep audit trails. Different goals, different timelines—often built on separate systems.

TigerGraph supports both real-time fraud scoring and retrospective AML investigation on the same platform, enabling collaboration while respecting operational boundaries.

*Bonus challenge: Graph tools weren't built for production* Many graph projects stall because academic tools don't scale, notebooks don't deploy, and prototype logic doesn't hold up in live environments.

GSQL, TigerGraph's graph-native query language, supports reusable logic, custom scoring, and enterprise deployment. It's built for production fraud defense, not for theory.

## **Key Features of a High-Performance Fraud Detection Graph Database**

To detect fraud effectively in today's digital landscape, your graph platform needs more than storage and search. It must reason through relationships at speed, scale with complexity, and support real-time decision-making. TigerGraph delivers on all fronts.

*Real-Time, In-Graph Scoring*

Fraud detection should happen where the data lives—not in disconnected scoring engines.

With TigerGraph, risk scores are computed inside the graph, during query execution. This means you can flag suspicious behavior the moment it happens—without exporting data or waiting for batch jobs.

### *Massively Parallel Multi-Hop Traversal*

Sophisticated fraud doesn't stop at one connection. You may need to explore five or six hops across accounts, IPs, devices, and merchants. TigerGraph handles this with parallel traversal across distributed compute clusters—returning results in milliseconds, even with billions of edges.

### *Built-In and Customizable Graph Algorithms*

Different fraud patterns require different detection logic—centrality for influence, community detection for collusion, and similarity for identity. TigerGraph provides production-grade algorithms out of the box. Need something custom? Extend or compose new algorithms using GSQL.

### *Streaming Ingestion and Schema Evolution*

Fraud patterns shift fast. Your graph should, too. TigerGraph supports real-time data ingestion from APIs, logs, and event streams—ensuring your graph reflects the current state of play. Schema evolution lets you model new behaviors (like biometric signals or third-party breach exposure) without downtime.

### *REST APIs and Embedded Intelligence*

Detection is only valuable if it's actionable. TigerGraph exposes query results and scoring logic via RESTful APIs, enabling seamless integration into login systems, payment gateways, fraud dashboards, or SOC workflows. The graph becomes part of your real-time defense—not just your back-end analysis.

### *Battle-Tested Performance*

TigerGraph is deployed at a global scale in fraud environments where milliseconds matter.

Customers run multi-hop queries in under 80ms and traverse billions of relationships per second—powering real-time risk scoring and instant investigation.

## 10.DATA STRUCTURE USE IN OS/DBMS

### 1. PAGE REPLACEMENT ALGORITHMS IN OPERATING SYSTEMS

In an operating system that uses paging, a page replacement algorithm is needed when a page fault occurs and no free page frame is available. In this case, one of the existing pages in memory must be replaced with the new page.

The virtual memory manager performs this by:

1. Selecting a victim page using a page replacement algorithm.
2. Marking its page table entry as “not present.”
3. If the page was modified (dirty), writing it back to disk before replacement.

The efficiency of a page replacement algorithm directly affects the page fault rate, which in turn impacts system performance.

#### Common Page Replacement Techniques

- First In First Out (FIFO)
- Optimal Page replacement
- Least Recently Used (LRU)
- Most Recently Used (MRU)

#### • 1. First In First Out (FIFO)

This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

**Example 1:** Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3-page frames. Find the number of page faults using FIFO Page Replacement Algorithm.



- Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots ---> **3 Page Faults.**
- When 3 comes, it is already in memory so ---> **0 Page Faults.** • Then 5 comes, it is not available in memory, so it replaces the oldest page slot i.e 1. ---> **1 Page Fault.**
- 6 comes, it is also not available in memory, so it replaces the oldest page slot i.e 3 ---> **1 Page Fault.**
- Finally, when 3 come it is not available, so it replaces 0 1-page **fault.**

**Example -2.** Consider page reference string 1, 3, 0, 3, 5, 6 and 3 page slots. Initially all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots —> **3 Page Faults.**

- when 3 comes, it is already in memory so —> 0 Page Faults. Then 5 comes, it is not available in memory so it replaces the oldest page slot i.e 1. —> **1Page Fault.**
- Finally 6 comes, it is also not available in memory so it replaces the oldest page slot i.e 3 —> **6 Page Fault.**
- So total page faults = **5.**

• **Example -2.** Consider the following reference string: 0, 2, 1, 6, 4, 0, 1, 0, 3, 1, 2, 1. Using FIFO page replacement algorithm -



- So, total number of page faults = 9. Given memory capacity (as number of

pages it can hold) and a string representing pages to be referred, write a function to find number of page faults.

## 2. Optimal Page Replacement

In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

**Example:** Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find number of page fault using Optimal Page Replacement Algorithm.



### Optimal Page Replacement

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ---> 4 Page faults
- 0 is already there so ---> 0 Page fault. when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future---> 1 Page fault.
- 0 is already there so ---> 0 Page fault. 4 will takes place of 1 ---> 1 Page Fault.

Now for the further page reference string ---> 0 Page fault because they are already available in the memory. [Optimal page replacement](#) is perfect, but not possible in practice as the operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other

replacement algorithms can be analyzed against it.

### 3. Least Recently Used

In this algorithm, page will be replaced which is least recently used.

**Example** Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frames. Find number of page faults using LRU Page Replacement Algorithm.



#### Least Recently Used - Page Replacement

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ---> 4 Page faults.
- 0 is already there so ---> 0 Page fault. when 3 came it will take the place of 7 because it is least recently used ---> 1 Page fault.
- 0 is already in memory so ---> 0 Page fault.
- 4 will takes place of 1 ---> 1 Page Fault.
- Now for the further page reference string ---> 0 Page fault because they are already available in the memory.

### 4. Most Recently Used (MRU)

In this algorithm, page will be replaced which has been used recently.

Belady's anomaly can occur in this algorithm.

**Example 4:** Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3,

2, 3 with 4-page frames. Find number of page faults using MRU Page Replacement Algorithm.



#### Most Recently Used - Page Replacement

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots  
---> 4 Page faults
- 0 is already there so--> 0 page fault
- when 3 comes it will take place of 0 because it is most recently used ---> 1 Page fault
- when 0 comes it will take place of 3 ---> 1 Page fault
- when 4 comes it will take place of 0 ---> 1 Page fault
- 2 is already in memory so ---> 0 Page fault
- when 3 comes it will take place of 2 ---> 1 Page fault
- when 0 comes it will take place of 3 ---> 1 Page fault
- when 3 comes it will take place of 0 ---> 1 Page fault
- when 2 comes it will take place of 3 ---> 1 Page fault
- when 3 comes it will take place of 2 ---> 1 Page fault

### **INDEXING IN DATABASES**

Indexing in DBMS is used to speed up data retrieval by minimizing disk scans. Instead of searching through all rows, the DBMS uses index structures to quickly locate data using key values.

When an index is created, it stores sorted key values and pointers to actual data rows. This reduces the number of disk accesses, improving performance especially

on large datasets.



### Attributes of Indexing

Several Important attributes of indexing affect the performance and efficiency of database operations:

1. **Access Types:** This refers to the type of access such as value-based search, range access, etc.
2. **Access Time:** It refers to the time needed to find a particular data element or set of elements.
3. **Insertion Time:** It refers to the time taken to find the appropriate space and insert new data.
4. **Deletion Time:** Time taken to find an item and delete it as well as update the index structure.
5. **Space Overhead:** It refers to the additional space required by the index.



## **File Organization in Indexing**

File organization refers to how data and indexes are physically stored in memory or on disk. The following are the common types of file organizations used in indexing:

### **1. Sequential (Ordered) File Organization**

In this type of organization, the indices are based on a sorted ordering of the values. These are generally fast and a more traditional type of storing mechanism. These Ordered or Sequential file organizations might store the data in a dense or sparse format.

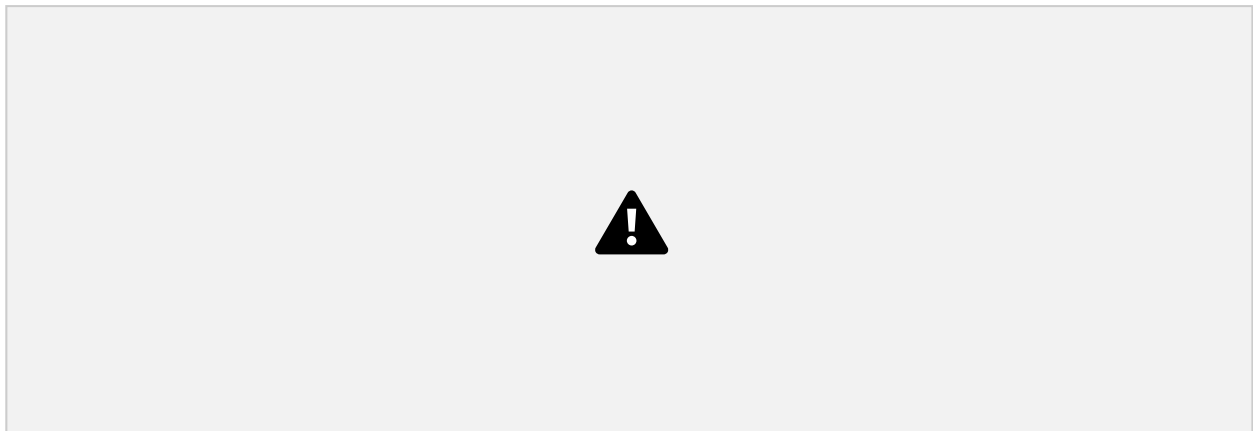
**i. Dense Index:** Every search key value in the data file corresponds to an index record. This method ensures that each key value has a reference to its data location.

**Example:** If a table contains multiple entries for the same key, a dense index ensures that each key value has its own index record.



**ii. Sparse Index:** The index record appears only for a few items in the data file. Each item points to a block as shown. To locate a record, we find the index record with the largest search key value less than or equal to the search key value we are looking for.

**Access Method:** To locate a record, we find the index record with the largest key value less than or equal to the search key, and then follow the pointers sequentially.



## 2. Hash File Organization

Uses a hash function to map keys to buckets.

- Offers fast access for exact-match queries.
- Not suitable for range queries.

### Types of Indexing Methods

There are different types of indexing techniques, each optimized for specific use

cases.

## 1. Clustered Indexing

**Clustered Indexing** stores related records together in the same file, reducing search time and improving performance, especially for join operations. Data is stored in sorted order based on a key (often a **non-primary key**) to group similar records, like students by semester. If the indexed column isn't unique, multiple columns can be combined to form a unique key. This makes data retrieval faster by keeping related records close and allowing quicker access through the index.



## 2. Primary Indexing

This is a type of Clustered Indexing wherein the data is sorted according to the search key and the **primary key** of the database table is used to create the index. It is a default format of indexing where it induces sequential file organization. As primary keys are **unique** and are stored in a sorted manner, the performance of the **searching operation** is quite efficient.

**Key Features:** The data is stored in sequential order, making searches faster and more efficient.

## 3. Non-clustered or Secondary Indexing

A non-clustered index just tells us where the data lies, i.e. it gives us a list of virtual pointers or references to the location where the data is actually stored. Data is not physically stored in the order of the index. Instead, data is present in leaf nodes.

**Example:** The contents page of a book. Each entry gives us the page number or location of the information stored. The actual data here (information on each page of

the book) is not organized but we have an ordered reference(contents page) to where the data points actually lie. We can have only dense ordering in the non-clustered index as sparse ordering is not possible because data is not physically organized accordingly.

It requires more time as compared to the clustered index because some amount of extra work is done in order to extract the data by further following the pointer. In the case of a clustered index, data is directly present in front of the index.



#### 4. Multilevel Indexing

With the growth of the size of the database, indices also grow. As the index is stored in the **main memory**, a single-level index might become too large a size to store with **multiple disk accesses**. The multilevel indexing segregates the main block into various smaller blocks so that the same can be stored in a single block.

The outer blocks are divided into **inner blocks** which in turn are pointed to the data blocks. This can be easily stored in the main memory with fewer overheads. This hierarchical approach reduces memory overhead and speeds up query execution.



#### Advantages of Indexing

- **Faster Queries:** Indexes allow quick search of rows matching specific values, speeding up data retrieval.
  - **Efficient Access:** Reduces disk I/O by keeping frequently accessed data in memory.
  - **Improved Sorting:** Speeds up sorting by indexing the relevant columns. •
- Consistent Performance:** Maintains query speed even as data grows.
- **Data Integrity:** Ensures uniqueness in columns indexed as unique, preventing duplicate entries.

### Disadvantages of Indexing

While indexing offers many advantages, it also comes with certain trade-offs:

- **Increased Storage Space:** Indexes require additional storage. Depending on the size of the data, this can significantly increase the overall storage requirements.
- **Increased Maintenance Overhead:** Indexes must be updated whenever data is inserted, deleted, or modified, which can slow down these operations.
- **Slower Insert/Update Operations:** Since indexes must be maintained and updated, inserting or updating data takes longer than in a non-indexed database.
- **Complexity in Choosing the Right Index:** Determining the appropriate indexing strategy for a particular dataset can be challenging and requires an understanding of query patterns and access behaviors.

### Features of Indexing

Several key features define the indexing process in databases:

- **Efficient Data Structures:** Indexes use efficient data structures like B-trees, B+ trees, and hash tables to enable fast data retrieval.
- **Periodic Index Maintenance:** Indexes need to be periodically maintained, especially when the underlying data changes frequently. Maintenance tasks include updating, rebuilding, or removing obsolete indexes.

- **Query Optimization:** Indexes play a critical role in query optimization. The DBMS query optimizer uses indexes to determine the most efficient execution plan for a query.
- **Handling Fragmentation:** Index fragmentation can reduce the effectiveness of an index. Regular defragmentation can help maintain optimal performance.